

6.S974 Games, Learning, and Security

Erich Trieschman

September 25, 2026

1. Relief logistics.

- (a) *Min-cost flow problem.* Noticing that there's more available supply than demand, we introduce a "surplus" node s that demands $100 = 1000 - 900$ to balance the problem.

Constants

$$\begin{aligned} b &= [b_{c1}, b_{c2}, b_w, b_{d1}, b_{d2}, b_s] \\ &= [-200, -700, 0, 400, 600, -100] \in \mathbb{R}^6 \\ c &= [c_{d1 \rightarrow c1}, c_{d1 \rightarrow c2}, c_{d1 \rightarrow w}, c_{d2 \rightarrow c1}, c_{d2 \rightarrow c2}, c_{d2 \rightarrow w}, c_{w \rightarrow c1}, c_{w \rightarrow c2}, c_{d1 \rightarrow s}, c_{d2 \rightarrow s}] \\ &= [30, 30, 15, 50, 23, 15, 11, 14, 0, 0] \in \mathbb{R}^{10} \end{aligned}$$

Variable

$$f = [f_{d1 \rightarrow c1}, f_{d1 \rightarrow c2}, f_{d1 \rightarrow w}, f_{d2 \rightarrow c1}, f_{d2 \rightarrow c2}, f_{d2 \rightarrow w}, f_{w \rightarrow c1}, f_{w \rightarrow c2}, f_{d1 \rightarrow s}, f_{d2 \rightarrow s}] \in \mathbb{R}^{10}$$

Linear program

$$\begin{aligned} \min_f \quad & 30f_{d1 \rightarrow c1} + 30f_{d1 \rightarrow c2} + 15f_{d1 \rightarrow w} + 50f_{d2 \rightarrow c1} + 23f_{d2 \rightarrow c2} \\ & + 15f_{d2 \rightarrow w} + 11f_{w \rightarrow c1} + 14f_{w \rightarrow c2} + 0f_{d1 \rightarrow s} + 0f_{d2 \rightarrow s} \\ \text{s.t.} \quad & f_{d1 \rightarrow c1} + f_{d2 \rightarrow c1} + f_{w \rightarrow c1} = 200 && (\text{Community 1}) \\ & f_{d1 \rightarrow c2} + f_{d2 \rightarrow c2} + f_{w \rightarrow c2} = 700 && (\text{Community 2}) \\ & f_{d1 \rightarrow w} + f_{d2 \rightarrow w} = f_{w \rightarrow c1} + f_{w \rightarrow c2} && (\text{Warehouse}) \\ & 400 = f_{d1 \rightarrow c1} + f_{d1 \rightarrow c2} + f_{d1 \rightarrow w} + f_{d1 \rightarrow s} && (\text{Depot 1}) \\ & 600 = f_{d2 \rightarrow c1} + f_{d2 \rightarrow c2} + f_{d2 \rightarrow w} + f_{d2 \rightarrow s} && (\text{Depot 2}) \\ & f_{d1 \rightarrow s} + f_{d2 \rightarrow s} = 100 && (\text{Surplus}) \\ & f \succeq 0 && (\text{Nonnegativity}) \end{aligned}$$

Directed network drawing. See Figure 1

- (b) *Fully loaded containers.* Yes the problem above admits a solution satisfying the constraint on fully loaded containers:

1. Scale the problem so supply, flows, demand, and costs are per 10 units:

$$b = [-20, -70, 0, 40, 60, -10], \quad c = [300, 300, 150, 500, 230, 150, 110, 140, 0, 0]$$

2. Integrality theorem says min-cost flow problem admits an integer optimal solution if i) the problem has an optimal and ii) $(b, u) \in \mathcal{Z}^{|n| \cdot |f|}$. Our problem satisfies i) as seen in part (d) and ii) since $b \in \mathcal{Z}^{|n|}$ is satisfied after problem scaling and since flows are only restricted to be nonnegative.

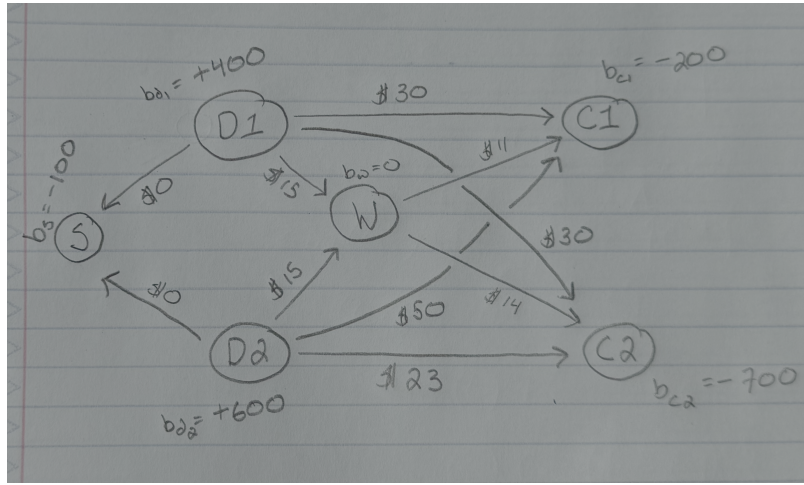


Figure 1: Problem 1 Part (a) directed network drawing

3. An integer solution after scaling represents a solution in increments of 10, i.e., fully loaded containers.
- (c) *Extreme events.* First we introduce new variables x_{c1}, x_{c2} to capture the amount of supply delivered to communities 1 and 2 respectively. We add corresponding constraints to limit delivered supply to be non-negative and to not exceed community demand. We then formulate the objective of maximum unmet demand, defined as

$$\max \left\{ 1 - \frac{x_{c1}}{500}, 1 - \frac{x_{c2}}{700} \right\}.$$

Then we introduce epigraph variable z to reformulate the maximum in the objective as epigraph constraints. Finally, we add additional constraints on the shipment budget and capacitated flows from the warehouse to community 1. The final linear program is

$$\begin{aligned}
 \min_{f, x, z} \quad & z \\
 \text{s.t.} \quad & f_{d1 \rightarrow c1} + f_{d2 \rightarrow c1} + f_{w \rightarrow c1} = x_{c1} && \text{(Community 1)} \\
 & f_{d1 \rightarrow c2} + f_{d2 \rightarrow c2} + f_{w \rightarrow c2} = x_{c2} && \text{(Community 2)} \\
 & f_{d1 \rightarrow w} + f_{d2 \rightarrow w} = f_{w \rightarrow c1} + f_{w \rightarrow c2} && \text{(Warehouse)} \\
 & 400 \geq f_{d1 \rightarrow c1} + f_{d1 \rightarrow c2} + f_{d1 \rightarrow w} && \text{(Depot 1; unused supply allowed)} \\
 & 600 \geq f_{d2 \rightarrow c1} + f_{d2 \rightarrow c2} + f_{d2 \rightarrow w} && \text{(Depot 2; unused supply allowed)} \\
 & f \geq 0 && \text{(Nonnegativity)} \\
 & f_{w \rightarrow c1} \leq 200 && \text{(Capacitated flow)} \\
 & 0 \leq x_{c1} \leq 500, 0 \leq x_{c2} \leq 700 && \text{(Deliveries can't exceed demand)} \\
 & 30f_{d1 \rightarrow c1} + 30f_{d1 \rightarrow c2} + 15f_{d1 \rightarrow w} + 50f_{d2 \rightarrow c1} + 23f_{d2 \rightarrow c2} + 15f_{d2 \rightarrow w} + 11f_{w \rightarrow c1} + 14f_{w \rightarrow c2} \leq 23000 && \text{(Budget constraint)} \\
 & 1 - \frac{1}{500}x_{c1} \leq z, 1 - \frac{1}{700}x_{c2} \leq z && \text{(Reformulated max in objective)}
 \end{aligned}$$

- (d) *In code.* Supply can remain unused while demand is unmet because constraints on the total budget and on flows between the warehouse and community 1. Specifically, restricting $w \rightarrow c1$ redirects flows to either $d1 \rightarrow c1$ at an incremental cost of $+4 = 30 - (15 + 11)$ or $d2 \rightarrow c1$ at an incremental cost of $+24 = 50 - (15 + 11)$. Relaxing either the budget or the flow constraint would allow us to meet more unmet demand with our unused supply.

```

import numpy as np
import cvxpy as cp

# =====
# Problem 1 Part (a)
# =====
# constants
nodes = np.array(["C1", "C2", "W", "D1", "D2", "S"])
arcs = np.array(
    [
        "D1->C1",
        "D1->C2",
        "D1->W",
        "D2->C1",
        "D2->C2",
        "D2->W",
        "W->C1",
        "W->C2",
        "D1->S",
        "D2->S",
    ]
)
b = np.array([-200, -700, 0, 400, 600, -100])
c = np.array([30, 30, 15, 50, 23, 15, 11, 14, 0, 0])
A = np.array(
    [
        [-1, 0, 0, 1, 0, 0],
        [0, -1, 0, 1, 0, 0],
        [0, 0, -1, 1, 0, 0],
        [-1, 0, 0, 0, 1, 0],
        [0, -1, 0, 0, 1, 0],
        [0, 0, -1, 0, 1, 0],
        [-1, 0, 1, 0, 0, 0],
        [0, -1, 1, 0, 0, 0],
        [0, 0, 0, 1, 0, -1],
        [0, 0, 0, 0, 1, -1],
    ]
)
# variable
f = cp.Variable(len(arcs))
# problem
objective = cp.Minimize(c @ f)
constraints = [A.T @ f == b, f >= 0]
problem = cp.Problem(objective, constraints)
problem.solve()
# solution
print("~~~ PROBLEM 1 PART (A) ~~~")
print("Optimal shipping plan:")
for arc, flow in zip(arcs, np.round(f.value, 2)):
    print(f"    {arc:>6}: {flow:8.2f}")
print("Optimal cost:", np.round(problem.value, 2))

# =====
# Problem 1 Part (c)
# =====
# constants
nodes_c = nodes[:-1]
arcs_c = arcs[:-2]
b_c = np.array([-500, -700, 0, 400, 600])
c_c = c[:-2]
A_c = A[:-2, :-1]
budget = 23_000
# variables
f_c = cp.Variable(len(arcs_c))
x = cp.Variable(2)
z = cp.Variable()
# problem
objective = cp.Minimize(z)
inj = A_c.T @ f_c
constraints = [

```

```

    inj[0:2] + x == 0, # LHS is outflow - inflow
    inj[2] == b_c[2],
    inj[3:] <= b_c[3:],
    f_c >= 0,
    x >= 0,
    x <= -b_c[0:2],
    f_c[6] <= 200,
    c_c @ f_c <= budget,
    1 - x / (-b_c[0:2]) <= z,
]
problem = cp.Problem(objective, constraints)
problem.solve()
print("\n~~~ PROBLEM 1 PART (C) ~~~")
print("Optimal shipping plan:")
for arc, flow in zip(arcs_c, np.round(f_c.value, 2)):
    print(f"    {arc:>6}: {flow:8.2f}")
print("Shipping cost:", np.round(c_c @ f_c.value, 2))
print("Unmet demand (C1, C2):", np.round(-b_c[:2] - x.value, 2))
print("Unmet fractions (C1, C2):", np.round(1 - x.value / (-b_c[0:2]), 2))
print("Unused supply (D1, D2):", np.round(b_c[3:] - inj[3:].value, 2))

# ~~~ PROBLEM 1 PART (A) ~~~
# Optimal shipping plan:
# D1->C1:    0.00
# D1->C2:    0.00
# D1->W:    300.00
# D2->C1:    0.00
# D2->C2:    600.00
# D2->W:    0.00
# W->C1:    200.00
# W->C2:    100.00
# D1->S:    100.00
# D2->S:    0.00
# Optimal cost: 21900.0

# ~~~ PROBLEM 1 PART (C) ~~~
# Optimal shipping plan:
# D1->C1:    182.64
# D1->C2:    0.00
# D1->W:    168.03
# D2->C1:    0.00
# D2->C2:    535.69
# D2->W:    31.97
# W->C1:    200.00
# W->C2:    0.00
# Shipping cost: 23000.0
# Unmet demand (C1, C2): [117.36 164.31]
# Unmet fractions (C1, C2): [0.23 0.23]
# Unused supply (D1, D2): [49.33 32.34]

```

2. Security Inspections in a Shared Building

- (a) *Strictly dominated actions.* For both companies, choosing to inspect the lobby L strictly dominates choosing to inspect their own storage room S_i . By symmetry, we only need to look at company A and can extrapolate to B:

$$u_A(L, O_B) = 3 > 1 = u_A(S_A, O_B), \quad u_A(L, L) = 5 > 4 = u_A(S_A, L), \quad u_A(L, S_B) = 3 > 2 = u_A(S_A, S_B).$$

A strictly dominated strategy can be dropped from our analysis, so we can drop choice S_i for both companies.

- (b) *Pure strategy Nash equilibria.* There are two pure strategy Nash equilibria $(s_A, s_B) \in \{(L, O_B), (O_A, L)\}$. Payoffs are (3, 6) and (6, 3) respectively.
- (c) *Nash equilibria with randomizations.*

No single-player randomization. First note that all mixed strategies require both companies to randomize: Suppose company A randomizes and company B plays pure $\{O_B, L\}$. Against O_B

company A's unique best response is L , and against L company A's unique best response is O_A , so company A's best response is a pure strategy, contradicting the supposition. By symmetry the reverse of company B and A holds.

Candidate Nash equilibria. Assume there exists a fully mixed Nash equilibrium (σ_A^*, σ_B^*) , then

$$\begin{aligned} \text{supp}(\sigma_A^*) = \{O_A, L\} \subseteq BR_A(\sigma_B^*) &\implies u_A(O_A, \sigma_B^*) = u_A(L, \sigma_B^*) \\ &\implies \mathbb{E}_{s_B \sim \sigma_B^*}[u_A(O_A, s_B)] = \mathbb{E}_{s_B \sim \sigma_B^*}[u_A(L, s_B)] \\ &\implies 1 \cdot (1 - \sigma_B^*(L)) + 6 \cdot \sigma_B^*(L) = 3 \cdot (1 - \sigma_B^*(L)) + 5 \cdot \sigma_B^*(L) \\ &\implies \sigma_B^*(L) = \frac{2}{3}, \sigma_B^*(O_B) = \frac{1}{3} \end{aligned}$$

And by symmetry in the payoff structure we also have $\sigma_A^*(L) = \frac{2}{3}$, $\sigma_A^*(O_A) = \frac{1}{3}$. Each indifference condition is a linear equation in one unknown with a unique solution, so this is the only candidate.

Verify. Finally, verify with

$$\sigma^* \in \Sigma \text{ is Nash equilibrium} \iff u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(s_i, \sigma_{-i}^*) \forall s_i \in S_i, \forall i \in \mathcal{I}$$

$$\begin{aligned} u_A(\sigma_A^*, \sigma_B^*) &= \mathbb{E}_{s_A \sim \sigma_A^*, s_B \sim \sigma_B^*}[u_A(s_A, s_B)] \\ &= \mathbb{E}_{s_A \sim \sigma_A^*} \left[\mathbb{E}_{s_B \sim \sigma_B^*}[u_A(s_A, s_B)] \right] \quad (\sigma_A^* \text{ independent from } \sigma_B^*) \\ &= \frac{1}{3} \mathbb{E}_{s_B \sim \sigma_B^*}[u_A(O_A, s_B)] + \frac{2}{3} \mathbb{E}_{s_B \sim \sigma_B^*}[u_A(L, s_B)] \\ &= \frac{1}{3} \left(\frac{1}{3} + 6 \cdot \frac{2}{3} \right) + \frac{2}{3} \left(3 \cdot \frac{1}{3} + 5 \cdot \frac{2}{3} \right) \quad (\text{plugging in from above}) \\ &= \frac{1}{3} \cdot \frac{13}{3} + \frac{2}{3} \cdot \frac{13}{3} = \frac{13}{3} \end{aligned}$$

From above we know $S_A = \{O_A, L\}$ and $u_A(O_A, \sigma_B^*) = u_A(L, \sigma_B^*) = \frac{13}{3}$. By symmetry the same holds for B, with $u_B(\sigma_A^*, \sigma_B^*) = \frac{13}{3}$. Hence (σ_A^*, σ_B^*) with $\sigma_A^*(O_A) = \sigma_B^*(O_B) = \frac{1}{3}$ is a Nash equilibrium

(d) *Correlated equilibria.*

Constraints explained.

- Equilibrium: no player gains by deviating from their recommendation, given the other follows theirs in probability

$$\begin{aligned} \sum_{s_B \in S_B} \pi(s_A, s_B)[u_A(s_A, s_B) - u_A(s'_A, s_B)] &\geq 0, \quad \forall s_A, s'_A \in S_A \\ \sum_{s_A \in S_A} \pi(s_A, s_B)[u_B(s_B, s_A) - u_B(s'_B, s_A)] &\geq 0, \quad \forall s_B, s'_B \in S_B \end{aligned}$$

- Joint distribution: nonnegativity of probabilities, and probabilities sum to 1

$$\pi(s_A, s_B) \geq 0 \forall (s_A, s_B) \in S_A \times S_B, \quad \sum_{s_A \in S_A, s_B \in S_B} \pi(s_A, s_B) = 1$$

Eliminations. Strictly dominated actions identified in 2(a) receive zero probability in any correlated equilibrium and can be dropped. Suppose s'_i is strictly dominated by s_i , i.e. $u_i(s_i, s_{-i}) > u_i(s'_i, s_{-i})$ for all $s_{-i} \in S_{-i}$. The equilibrium constraint for recommendation s'_i requires

$$\sum_{s_{-i} \in S_{-i}} \pi(s'_i, s_{-i})[u_i(s'_i, s_{-i}) - u_i(s_i, s_{-i})] \geq 0$$

Since bracketed term is strictly negative and $\pi(s'_i, s_{-i}) \geq 0$, this holds only if $\pi(s'_i, s_{-i}) = 0$ for all s_{-i} .

Instantiated.

$$\begin{aligned}
& \max_{\pi} \quad \pi(O_A, O_B)[1 + 1] + \pi(O_A, L)[6 + 3] + \pi(L, O_B)[3 + 6] + \pi(L, L)[5 + 5] \\
& \text{s.t.} \quad \pi(O_A, O_B)[1 - 3] + \pi(O_A, L)[6 - 5] \geq 0 \\
& \quad \pi(L, O_B)[3 - 1] + \pi(L, L)[5 - 6] \geq 0 \\
& \quad \pi(O_A, O_B)[1 - 3] + \pi(L, O_B)[6 - 5] \geq 0 \\
& \quad \pi(O_A, L)[3 - 1] + \pi(L, L)[5 - 6] \geq 0 \\
& \quad \pi(s_A, s_B) \geq 0, \forall s_A \in \{O_A, L\}, \forall s_B \in \{O_B, L\} \\
& \quad \pi(O_A, O_B) + \pi(O_A, L) + \pi(L, O_B) + \pi(L, L) = 1
\end{aligned}$$

In code.

```

import numpy as np
import cvxpy as cp

# =====
# Problem 2 Part (d)
# =====
# Constants
U = np.array([[1, 1], [6, 3]], [[3, 6], [5, 5]])
A, B = 0, 1
O, L = 0, 1

# variable
pi = cp.Variable((2, 2))

# constraints
constraints = [pi >= 0, cp.sum(pi) == 1]
constraints += [
    cp.sum([pi[s_i, s_mi] * (U[s_i, s_mi, A] - U[sp_i, s_mi, A])
            for s_mi in [O, L]]
    ) >= 0
    for sp_i in [O, L]
    for s_i in [O, L]
]
constraints += [
    cp.sum([pi[s_mi, s_i] * (U[s_mi, s_i, B] - U[s_mi, sp_i, B])
            for s_mi in [O, L]]
    ) >= 0
    for sp_i in [O, L]
    for s_i in [O, L]
]

# solve problem
objective = cp.Maximize(cp.sum(cp.multiply(pi, U[:, :, A] + U[:, :, B])))
problem = cp.Problem(objective, constraints)
problem.solve()

# report
print("\n~~~ PROBLEM 2 PART (D) ~~~")
print("Joint distribution (pi):\n", np.round(pi.value, 2))
ep_A = cp.sum(cp.multiply(pi, U[:, :, A])).value
ep_B = cp.sum(cp.multiply(pi, U[:, :, B])).value
print(f"Expected payoffs (A, B): ({ep_A:0.2f}, {ep_B:0.2f})")
print("Total expected welfare:", np.round(ep_A + ep_B, 2))

# ~~~ PROBLEM 2 PART (D) ~~~
# Joint distribution (pi):
# [[-0.    0.25]
# [ 0.25  0.5 ]]
# Expected payoffs (A, B): (4.75, 4.75)
# Total expected welfare: 9.5

```

Here I notice that the expected payoffs in a correlated equilibrium are greater than expected payoffs in mixed strategy Nash equilibrium, $4.75 > 4.33$.

(e) *Building manager announces both recommendations.* No, π_{CE} would not remain in equilibrium:

- In a correlated equilibrium, company i has a shared view of the distribution of joint actions π_{CE} but only controls its own action s_i .
- Mediation under a joint distribution is in equilibrium if neither company is incentivized to deviate from its private recommendation. This works because each player i holds an assumption about other player's action choice s_{-i} : $\pi_{CE}(s_{-i} | s_i)$. And the conditional probability is such that company i is not incentivized to deviate from its recommendation s_i :

$$\sum_{s_{-i}} \pi_{CE}(s_{-i} | s_i) \cdot u_i(s_i, s_{-i}) \geq \sum_{s_{-i}} \pi_{CE}(s_{-i} | s_i) \cdot u_i(s'_i, s_{-i}) \quad \forall s'_i$$

These are exactly the constraints in our LP.

- A mediator announcing both recommendations publicly collapses the conditional mixed strategy assumption about company $-i$ held by company i : $\pi_{CE}(s_{-i} | s_i)$ to the pure strategy action s_{-i}
- Under the pure strategy action of company $-i$, company i may no longer be incentivized to follow its recommendation s_i , destroying the equilibrium.
- Using numbers from our specific case, suppose the mediator announces (L, L) to both companies. If company A assumes company B inspects L , it will have the incentive to deviate and inspect O_A for an increase in payoff of $5 \rightarrow 6$.

3. Human-AI Delegation.

(a) *Extensive form game tree.* See Figure 2

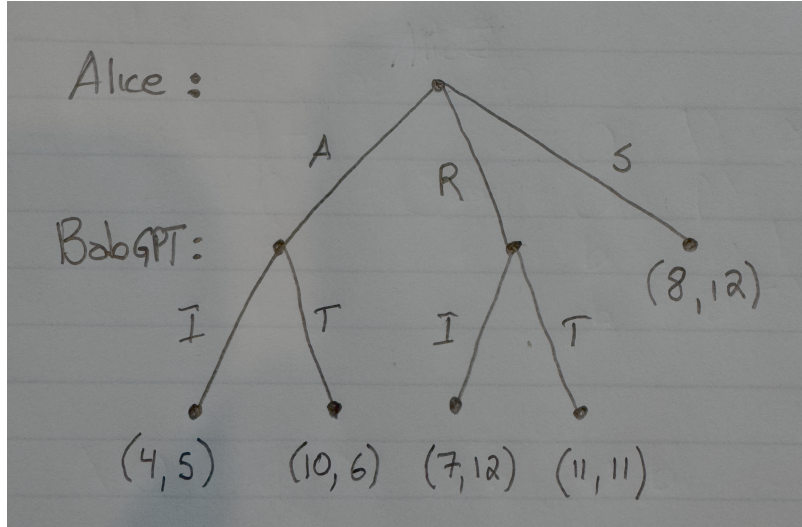


Figure 2: Problem 3 Part (a) directed network drawing

(b) *Pure strategies and subgame perfect equilibria.* Using first letter of each action for notation, and ordering BobGPTs actions following $A - R$; under Self, the AI does not act:

$$s_{\text{Alice}} \in \{A, R, S\}$$

$$s_{\text{BobGPT}} \in \{II, IT, TI, TT\}$$

SPE calculated through backward induction for $\{\Gamma(A), \Gamma(R), \Gamma\}$:

$\Gamma(A) : T|_A$ with payoffs (10, 6)

$\Gamma(R) : I|_R$ with payoffs (7, 12)

$\Gamma : A$ with payoffs (10, 6)

So (A, TI) is the SPE with payoffs (10, 6). The SPE is unique because no payoff was a tie.

(c) *Strategic form payoff matrix.*

	<i>II</i>	<i>IT</i>	<i>TI</i>	<i>TT</i>
<i>A</i>	(4, 5)	(4, 5)	(10, 6)	(10, 6)
<i>R</i>	(7, 12)	(11, 11)	(7, 12)	(11, 11)
<i>S</i>	(8, 12)	(8, 12)	(8, 12)	(8, 12)

Pure strategy Nash equilibria $\{(S, II), (A, TI)\}$:

- (S, II) : BobGPT chooses *II* $\implies$ Alice chooses *S* $\implies$ BobGPT doesn't deviate. All other BobGPT choices incentivize Alice to deviate from *S*.
- (A, TI) : BobGPT chooses *TI* $\implies$ Alice chooses *A* $\implies$ BobGPT doesn't deviate. All other BobGPT choices incentivize Alice to deviate from *A*.
- No equilibria with Alice choosing *R*: Alice chooses *R* $\implies$ BobGPT chooses $\{II, TI\} \implies$ Alice chooses $\{S, A\}$ respectively $\implies$ the equilibria above.
- No equilibria with BobGPT choosing *TT*: BobGPT chooses *TT* $\implies$ Alice chooses *R* $\implies$ see previous bullet.

(d) *Subgame perfect Nash equilibria.* No, not every Part(c) Nash equilibrium is subgame perfect:

- $s = (A, TI)$ is by Part(b).
- $s = (S, II)$ is not since $s|_A = I$ where *T* would lead to a higher payoff (6 > 5). Therefore s is not a Nash equilibrium of subgame $\Gamma(A)$ so it is not subgame perfect by definition.

(e) *Thinking after review and subgame perfect equilibria.* Thinking after Review fixes Review's payoff to (11, 11); BobGPT does not have a decision. Strategy profiles are

$$s_{\text{Alice}} \in \{A, R, S\}$$

$$s_{\text{BobGPT}} \in \{I, T\}$$

Subgames are $\{\Gamma(A), \Gamma\}$, and through backward induction

$\Gamma(A) : T|_A$ with payoffs (10, 6)

$\Gamma : R$ with payoffs (11, 11)

So (R, T) is the SPE with payoffs (11, 11). The SPE is unique because no payoff was a tie. Both players benefit relative to (A, TI) in Part(b) (Alice: 10 $\rightarrow$ 11; BobGPT: 6 $\rightarrow$ 11).

A non-binding promise of BobGPT to use Thinking after Review would not sustain this outcome: Under Alice's choice of Review, BobGPT's choice of Instant dominates Thinking (12 > 11). And anticipating BobGPT's choice of Instant after Review, Alice's choice of Ask dominates her choice of Review (10 > 7).

4. Autonomous pricing. *[Not covered yet.]*

- Strategic form game.*
- Subgame perfect equilibria.*
- New strategy profile.*
- Permanent punishment.*